

NLA Note

1 Background

Floating Point ($\mathbb{F}$): A number = mantissa $\times$ base^{exponent} (e.g. $2048 = 1 \times 2^{11}$ mantissa=1, base=2, exponent=11)

Assumptions: There is $\varepsilon_m > 0$ (**machine epsilon**) **A1:** $\forall \alpha \in \mathbb{R}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $fl : \mathbb{R} \rightarrow \mathbb{F}$ by $fl(\alpha) = \alpha(1 + \varepsilon)$

A2: $\forall \alpha, \beta \in \mathbb{F}, \forall * \in \{+, -, \times, \div\}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $\alpha * \beta = (\alpha * \beta)(1 + \varepsilon)$

A1: The rounding error in α is relative to the size of α ($|\alpha - fl(\alpha)| < |\alpha|\varepsilon_m$) A2: The error in $\alpha * \beta$ is relative to the size of $\alpha * \beta$ ($|\alpha * \beta - \alpha * \beta| \leq |\alpha * \beta|\varepsilon_m$)

IEEE (Double Precision): 64-bit: denote $\mathbb{F}$. 1-bit: sign 52-bit: mantissa 11-bit: exponent (base 2). $\varepsilon_m \approx 2.22 \times 10^{-16}$

Cost of Algorithm (C): $C :=$ number of floating point operations (FLOPs) If $C(n) \leq \alpha f(n), \Rightarrow C(n) = \mathcal{O}(f(n))$. n is problem size

• We consider that: 交换/赋值 no cost (i.e. Permutation $\mathbf{P}$ is free), 比较两数 1 FLOPs, $\sqrt{\quad}$ 1 FLOPs.

Norms: $\|\mathbf{x}\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}, p \geq 1$ $\|\mathbf{x}\|_\infty := \max_{1 \leq i \leq n} |x_i|$ $\|\mathbf{x}\|_2 := (\sum_{i=1}^n |x_i|^2)^{1/2}$ $\|\mathbf{x}\|_1 := \sum_{i=1}^n |x_i|$

1. $\|\mathbf{x}\|_p \geq 0$ $\|\mathbf{x}\|_p = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$

1. $\|\mathbf{A}_{m \times n}\|_\infty := \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$ (maximum row sum)

2. $\|\alpha \mathbf{x}\|_p = |\alpha| \|\mathbf{x}\|_p$ for $\alpha \in \mathbb{R}$

2. $\|\mathbf{A}_{m \times n}\|_1 := \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$ (maximum column sum)

3. $\|\mathbf{x} + \mathbf{y}\|_p \leq \|\mathbf{x}\|_p + \|\mathbf{y}\|_p$ (Triangle inequality)

3. $\|\mathbf{A}_{m \times n}\|_2 := \sqrt{\rho(\mathbf{A}^T \mathbf{A})}$ $\rho(\mathbf{A}^T \mathbf{A}) = \max\{|\lambda| : \lambda \text{ is eigenvalue of } \mathbf{A}^T \mathbf{A}\}$

4. **Induced Norm:** $\|\mathbf{A}\|_p := \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_p}{\|\mathbf{x}\|_p}$

2 Methods for Systems of Linear Equations (SLE)

Ideal: We want to solve SLE: $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}_{n \times n}$ is invertible, $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$

2.1 Backward Stable | Condition Number | Error

Backward Stable: An algorithm to solve SLE is Backward Stable if: $\exists \alpha(n) > 0$ s.t. $\forall \mathbf{A}, \mathbf{b}$, the computed solution $\hat{\mathbf{x}}$ satisfies:

$$(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta \mathbf{b} \text{ and } \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta \mathbf{b}\|_p}{\|\mathbf{b}\|_p} \leq \alpha \varepsilon_m \quad \varepsilon_m : \text{machine epsilon.} \quad \text{ps: } \alpha \text{ 可以和问题规模 } n \text{ 相关, 但不能和 } \mathbf{A} \text{ 或 } \mathbf{b} \text{ 相关.}$$

Condition Number ($\kappa_p(\mathbf{A})$): $\kappa_p(\mathbf{A}) := \begin{cases} \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p \geq 1 & \text{if } \mathbf{A} \text{ is invertible} \\ \infty & \text{otherwise} \end{cases}$ $\begin{cases} \text{If } \kappa_p(\mathbf{A}) \text{ is large (e.g. } \kappa_p(\mathbf{A}) \geq 10^{12} \approx \varepsilon_m^{-1} \text{), } & \mathbf{A} \text{ is ill-conditioned.} \\ \text{If } \kappa_p(\mathbf{A}) \leq 10^4, & \mathbf{A} \text{ is well-conditioned.} \end{cases}$ ps: $\kappa_p(\mathbf{A})$ 衡量矩阵奇异性

1. For $\mathbf{A}_{n \times n}$ with real eigenvalues $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$ and corresponding eigenvectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$: ps: 对所有实对称矩阵都满足下面条件

If $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ are orthonormal set. Then $\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2 = \frac{|\lambda_1|}{|\lambda_n|} \geq 1$

Relative Error: $\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|_p}{\|\mathbf{x}\|_p}$ **Actual Error:** $\|\hat{\mathbf{x}} - \mathbf{x}\|_p$ ps: Relative error gives more information than actual error.

1. Let $\mathbf{Ax} = \mathbf{b}$ and $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$. If $\kappa_p(\mathbf{A}) \|\Delta \mathbf{A}\|_p < \|\mathbf{A}\|_p$ then $\frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \leq \frac{\kappa_p(\mathbf{A})}{1 - \kappa_p(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \cdot \frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}$. 此时的 $\frac{\|\Delta \mathbf{A}\|_p}{\|\mathbf{A}\|_p}$ 小代表 good stability.

2.2 LU Factorization | Gaussian Elimination (GE)

Def of LU: If $\mathbf{A}_{n \times n}$ is invertible, then $\exists \mathbf{L}$: 单位下三角, $\exists \mathbf{U}$: 可逆上三角. s.t. $\mathbf{A} = \mathbf{LU}$ (**unique**)

LU Factorization: By Gaussian elimination, $\mathbf{A}_{n \times n} = (\prod_{k=1}^{n-1} \mathbf{L}_k)^{-1} \mathbf{U} =: \mathbf{LU}$ $\mathbf{L}_k$: Lower triangular with 1s on the diagonal (by row operation on single column)

1. 如果 $\mathbf{L}$ 是单位下三角矩阵, 且只有第 k 列有非零元素在对角线下, 则 $\mathbf{L}^{-1}$ 矩阵是 $\mathbf{L}$ 对角线不动, 其余为相反数.

2. 如果 $\mathbf{L}_1, \mathbf{L}_2$ 矩阵是单位下三角矩阵, 且 $\mathbf{L}_1 \mathbf{L}_2$ 在第 $1 \sim k | k+1 \sim n$ 列有非零元素在对角线下, 则 $\mathbf{L}_1 \mathbf{L}_2$ 矩阵是他们的"拼接".

3. By LU factorization, $\mathbf{A} = \mathbf{LU} \Rightarrow$ By solving $\mathbf{Ly} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{y}$, we can solve SLE $\mathbf{Ax} = \mathbf{b}$. 用 FS 和 BS 算法

$$\text{e.g. (1)} \quad \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix} \quad (2) \quad \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 5 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 5 & 1 \end{bmatrix}$$

Error Analysis of GE: Gaussian elimination is an **unstable** algorithm. (i.e. floating point arithmetic can have a disastrous effect on the computed solution.)

2.3 LUPP | LUCP | GEPP | GECP

Permutation Matrix: For a permutation π , $\mathbf{P}_\pi := [\mathbf{e}_{\pi(1)}, \dots, \mathbf{e}_{\pi(n)}]$ ps: 写的时候是对 I , 第 i 行的 1 移动到第 $\pi(i)$ 行. 注意: 用另一方式定义的 $\mathbf{P}_\pi$ 会让后面 π 结论不一样

• 左边乘 $\mathbf{P}_\pi$: 交换行 ($\mathbf{P}_\pi$ 的行代表具体行的交换方法, π 也代表交换行的方法)

• 右边乘 $\mathbf{P}_\pi$: 交换列 ($\mathbf{P}_\pi$ 的列代表具体列的交换方法, π^{-1} 才代表交换列的方法)

• Proposition: $\mathbf{P}_\pi$ is orthogonal, i.e. $\mathbf{P}_\pi^T = \mathbf{P}_\pi^{-1}$ ($\mathbf{P}_\pi^T \mathbf{P}_\pi = \mathbf{I}$)

• 对于二阶置换矩阵 (**Involution**): $\mathbf{P}_\pi^2 = \mathbf{I}$, i.e. $\mathbf{P}_\pi^{-1} = \mathbf{P}_\pi$

• **GEPP:** 交换行使每次 $|u_{kk}|$ 最大 **GECP:** 交换行和列使每次 $|u_{kk}|$ 最大

ps: GEPP and GECP are both **stable algorithms** for SLE.

Backward Stable of GEPP: $\forall$ SLE sol $\hat{\mathbf{x}}$ by GEPP, $\exists \Delta \mathbf{A}$ s.t. $(\mathbf{A} + \Delta \mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$ and $\frac{\|\Delta \mathbf{A}\|_\infty}{\|\mathbf{A}\|_\infty} \leq p(n) 2^{n-1} \varepsilon_m$ $p(n)$ is a cubic polynomial

2.4 QR Method | Householder QR factorisation

QR Factorization: QR factorization of $\mathbf{A}_{n \times n}$ is $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}$ is orthogonal (i.e. $\mathbf{Q}^T = \mathbf{Q}^{-1}$) and $\mathbf{R}$ is non-singular upper triangular.

• Def of $\mathbf{Q}$ is orthogonal is equivalent to $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$. (i.e. columns of $\mathbf{Q}$ forming an orthonormal set.)

• **Orthogonal Projection:** $\text{proj}_{\mathbf{u}}(\mathbf{x}) = \frac{\mathbf{x} \cdot \mathbf{u}}{\|\mathbf{u}\|_2} \mathbf{u} = (\mathbf{u}^T \mathbf{x}) \mathbf{u}$ if $\|\mathbf{u}\|_2 = 1$. $\mathbf{x} = \text{proj}_{\mathbf{u}}(\mathbf{x})^{\text{parallel to } \mathbf{u}} + (\mathbf{x} - \text{proj}_{\mathbf{u}}(\mathbf{x}))^{\text{orthogonal to } \mathbf{u}}$

3 Algorithms

3.1 DP MV MM | LU FS BS | GE GEPP GECP | Max GS MGS SQR QR

DP Dot product $C = \sum_{i=1}^n 2 = 2n = \mathcal{O}(n)$

Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

Output: $s = \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T \mathbf{y}$

1: $s = 0$

2: for $i = 1, \dots, n$ do

3: $s = s + x_i y_i$

4: end for

MV Matrix-vector multiplication

$C = \sum_{i=1}^m 2n = 2nm = \mathcal{O}(mn)$

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$

Output: $\mathbf{y} = \mathbf{Ax}$

1: for $i = 1, \dots, m$ do

2: $y_i = \mathbf{a}_i^T \mathbf{x}$ # DP Algorithm

3: end for

MM Matrix-matrix multiplication
$C = \sum_{i=1}^m \sum_{j=1}^p 2n = 2mnp = \mathcal{O}(mnp)$
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$ Output: $\mathbf{C} = \mathbf{AB}$ 1: for $i = 1, \dots, m$ do 2: for $j = 1, \dots, p$ do 3: $c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$ # DP Algorithm 4: end for 5: end for

LU LU factorisation
$C(n) = \sum_{k=1}^{n-1} (n-k)(1+2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{A} = \mathbf{LU}$ 1: $\mathbf{L} = \mathbf{I}, \mathbf{U} = \mathbf{A}$ 2: for $k = 1, \dots, n-1$ do 3: for $j = k+1, \dots, n$ do 4: $l_{jk} = u_{jk}/u_{kk}$ 5: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 6: end for 7: end for

GE Gaussian elimination
$C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{Ax} = \mathbf{b}$ 1: Find LU factorisation of $\mathbf{A}$. # LU Algorithm 2: Solve $\mathbf{Ly} = \mathbf{b}$ using forward substitution. # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ using back substitution. # BS Algorithm

LUPP LU factorisation with partial pivoting (not unique)
$C(n) = \frac{2}{3}n^3 + n^2 - \frac{5}{3}n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PA} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $i \in \{k, \dots, n\}$ which maximizes $ u_{ik} $ # Max Algorithm 4: Exchange row $(u_{kk}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部行交换 7: for $j = k+1, \dots, n$ do 8: $l_{jk} = u_{jk}/u_{kk}$ 9: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 10: end for 11: end for

Max Finding index of maximum
$C(m) = m-1$
Input: $z \in \mathbb{R}^m$ Output: index i with $ z_i = \max_{j=1, \dots, m} z_j $ 1: $i = 1, \max = z_1 $ 2: for $j = 2, \dots, m$ do 3: if $ z_j > \max$ then 4: $i = j$ 5: $\max = z_j $ 6: end if 7: end for

SQR (SLE) via QR-Factorization
$C(n) = 2n^3 + 4n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{Ax} = \mathbf{b}$ 1: Find QR factorization of $\mathbf{A}$ # GS Algorithm 2: Solve $\mathbf{Qy} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$ 3: Solve $\mathbf{Rx} = \mathbf{y}$ using Algorithm BS # BS Algorithm

4 Appendix

4.1 Useful Theorems/Formulas

Sum Notation: $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$

4.2 Others

Where the error come from calulation:

- If your calculation involves 15 or more significant digits, you will likely encounter rounding errors. (Since $\varepsilon_m \approx 10^{-16}$ (relative accuracy), which is about 15 significant digits.)
- For GE Algorithm:** It is an unstable algorithm. e.g. $\mathbf{A} = [10^{-20} \ 1 \ \backslash \ 1 \ 1] \Rightarrow$ By **GE** Algorithm, $\hat{\mathbf{L}}\hat{\mathbf{U}} = \mathbf{A} + [0 \ 0 \ \backslash \ 0 \ -1]$ (error)

FS Forward substitution
$C(n) = \sum_{j=1}^n [2(j-1) + 1] = n^2 = \mathcal{O}(n^2)$
Input: $\mathbf{L} \in \mathbb{R}^{n \times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{Ly} = \mathbf{b}$ 1: for $j = 1, \dots, n$ do 2: $y_j = \frac{1}{l_{jj}} \left(b_j - \sum_{k=1}^{j-1} l_{jk} y_k \right)$ 3: end for

GEPP Gaussian elimination with partial pivoting
$C(n) = \frac{2}{3}n^3 + 3n^2 - \frac{5}{3}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ # numpy.linalg.solve use it Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PA} = \mathbf{LU}$ using Algorithm LUPP # LUPP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ by back substitution # BS Algorithm

BS Back substitution
$C(n) = \sum_{j=1}^n [2(n-j) + 1] = n^2 = \mathcal{O}(n^2)$
Input: $\mathbf{U} \in \mathbb{R}^{n \times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{Ux} = \mathbf{y}$ 1: for $j = n, \dots, 1$ do 2: $x_j = \frac{1}{u_{jj}} \left(y_j - \sum_{k=j+1}^n u_{jk} x_k \right)$ 3: end for

GECP Gaussian elimination with complete pivoting
$C(n) = n^3 + 3n^2 - 2n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PAQ} = \mathbf{LU}$ using Algorithm LU with complete pivoting 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Uz} = \mathbf{y}$ by back substitution # BS Algorithm 4: Compute $\mathbf{x} = \mathbf{Qz}$

LUCP LU factorisation with complete pivoting (not unique)
$C(n) = n^3 + n^2 - 2n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P}, \mathbf{Q} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PAQ} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}, \mathbf{Q} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $(i, j) \in \{(k, k), \dots, (n, n)\}$ which maximizes $ u_{ij} $ # Max Algorithm 4: Exchange row $(u_{kk}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 U 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 L 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部行交换 P 7: Exchange column $(u_{1k}, \dots, u_{nk})$ with $(u_{1j}, \dots, u_{nj})$ # 第 k 列和第 j 列的全部行交换 U 8: Exchange column $(q_{1k}, \dots, q_{nk})$ with $(q_{1j}, \dots, q_{nj})$ # 第 k 列和第 j 列的全部行交换 Q 9: for $m = k+1, \dots, n$ do 10: $l_{mk} = u_{mk}/u_{kk}$ 11: $(u_{mk}, \dots, u_{mn}) = (u_{mk}, \dots, u_{mn}) - l_{mk}(u_{kk}, \dots, u_{kn})$ 12: end for 13: end for

GS Gram-Schmidt
$C(n) = 2n^3 + n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$, with orthonormal columns $\mathbf{q}_1, \dots, \mathbf{q}_n$ $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{R} = \mathbf{0}$ 2: for $j = 1, \dots, n$ do 3: $\mathbf{q}_j = \mathbf{a}_j$ 4: for $k = 1, \dots, j-1$ do 5: $r_{kj} = \mathbf{q}_k^\top \mathbf{a}_j$ 6: $\mathbf{q}_j = \mathbf{q}_j - r_{kj} \mathbf{q}_k$ 7: end for 8: $r_{jj} = \ \mathbf{q}_j\ _2 = \sqrt{\mathbf{q}_j^\top \mathbf{q}_j}$ 9: $\mathbf{q}_j = \frac{\mathbf{q}_j}{r_{jj}}$ 10: end for

SQR (SLE) via QR-Factorization
$C(n) = 2n^3 + 4n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{Ax} = \mathbf{b}$ 1: Find QR factorization of $\mathbf{A}$ # GS Algorithm 2: Solve $\mathbf{Qy} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$ 3: Solve $\mathbf{Rx} = \mathbf{y}$ using Algorithm BS # BS Algorithm